Planejamento de Testes – API ServeRest

Objetivos de Teste

- Validar regras de negócio da API
  - Garantir que a lógica de negócio implementada na API funcione conforme o esperado.
  - Exemplo: Um usuário comum não deve conseguir cadastrar um produto.
- Garantir integridade de dados
  - Assegurar que os dados sejam armazenados e recuperados corretamente, sem corrupção ou perda.
  - Exemplo: Após cadastrar um produto, suas informações devem ser consistentes ao serem consultadas.
- Verificar controle de acesso e autenticação
  - Confirmar que apenas usuários autorizados possam acessar recursos específicos.
  - Testar o fluxo de login e a validação de tokens.
- Assegurar consistência de contratos (Swagger)
  - Validar se a implementação da API está alinhada com a documentação do Swagger/OpenAPI.
  - Verificar tipos de dados, campos obrigatórios, e formatos de requisição/resposta.

Escopo dos Testes

- Em Escopo
  - Login
    - Teste de autenticação de usuários (sucesso e falha).
    - Geração e validação de Bearer Token.
  - Usuários
    - Criação, leitura, atualização e exclusão de usuários.
    - Testes de permissões para diferentes perfis (administrador, comum).
  - Produtos
    - Criação, leitura, atualização e exclusão de produtos.
    - Testes de controle de estoque.
  - Carrinhos
    - Criação de carrinhos.
    - Adição e remoção de produtos do carrinho.
    - Conclusão e cancelamento de compras.
- Fora do Escopo
  - Front-end — A interface de usuário não será testada nesta fase.
  - Teste de Carga — Testes de performance e escalabilidade não serão realizados.
  - Testes de Segurança avançados — Análise de vulnerabilidades (ex: SQL Injection, XSS) não será o foco principal.

Endpoints da API

- Login
  - POST /login — Realizar autenticação de usuário.
- Usuários
  - GET /usuarios — Listar todos os usuários.
  - POST /usuarios — Cadastrar um novo usuário.
  - GET /usuarios/{id} — Buscar usuário por ID.
  - PUT /usuarios/{id} — Editar usuário existente.
  - DELETE /usuarios/{id} — Excluir usuário.
- Produtos
  - GET /produtos — Listar todos os produtos.
  - POST /produtos — Cadastrar um novo produto.
  - GET /produtos/{id} — Buscar produto por ID.
  - PUT /produtos/{id} — Editar produto existente.
  - DELETE /produtos/{id} — Excluir produto.
- Carrinhos
  - GET /carrinhos — Listar todos os carrinhos.
  - POST /carrinhos — Cadastrar um novo carrinho.
  - GET /carrinhos/{id} — Buscar carrinho por ID.
  - DELETE /carrinhos/concluir-compra — Finalizar a compra de um carrinho.
  - DELETE /carrinhos/cancelar-compra — Cancelar a compra e retornar produtos ao estoque.

Regras de Negócio

- Autenticação via Bearer Token
  - Todas as rotas protegidas exigem um token JWT válido no cabeçalho Authorization.
  - O token é obtido após o login bem-sucedido.
- Permissões por perfil (administrador / comum)
  - Usuários com perfil de administrador possuem acesso irrestrito.
  - Usuários comuns têm acesso limitado (ex: não podem gerenciar produtos).
- Unicidade de e-mail — Não é permitido o cadastro de múltiplos usuários com o mesmo endereço de e-mail.
- Controle de estoque
  - A quantidade de produtos no estoque deve ser atualizada ao adicionar/remover produtos de um carrinho ou concluir/cancelar uma compra.
  - Não é possível adicionar mais produtos do que o disponível em estoque.
- Integridade de carrinho e usuário
  - Um usuário não pode ser excluído se tiver um carrinho ativo.
  - Um produto não pode ser excluído se estiver em um carrinho ativo.

Riscos e Pontos Críticos

- Quebra de autorização
  - Usuários não autorizados acessando recursos restritos.
  - Exemplo: Usuário comum criando um produto ou acessando dados de outro usuário.
- Inconsistência de estoque
  - Estoque de produtos não sendo atualizado corretamente após operações de carrinho.
  - Exemplo: Produto vendido, mas ainda disponível no estoque.
- Exclusão indevida de recursos
  - Exclusão de usuários ou produtos que possuem dependências (ex: carrinho ativo).
  - Exemplo: Excluir um usuário que tem um carrinho pendente.
- Falhas em transações (concluir/cancelar compra)
  - Erros que comprometem a integridade da compra/cancelamento.
  - Exemplo: Compra concluída, mas produtos não deduzidos do estoque ou dinheiro não processado.

Estratégia de Testes

- Testes funcionais de API
  - Verificação da funcionalidade de cada endpoint conforme os requisitos.
  - Testes de fluxo de trabalho completo (ex: login -> criar produto -> listar produtos).
- Testes de contrato
  - Validação da conformidade da API com a especificação OpenAPI/Swagger.
  - Uso de ferramentas para verificar esquemas de requisição e resposta.
- Testes exploratórios baseados em risco
  - Exploração manual da API focada nos pontos de maior risco identificados.
  - Geração de cenários não previstos para descobrir falhas.
- Abordagem híbrida (estruturado + exploratório) — Combinação de testes roteirizados e pré-definidos com a liberdade da exploração.

Técnicas e Heurísticas

- Testes Funcionais baseados em regra de negócio
  - Criação de cenários de teste diretamente das regras de negócio.
  - Exemplo: Testar que um produto com estoque zero não pode ser adicionado ao carrinho.
- Testes de Contrato de API
  - Uso de ferramentas como Postman, Newman ou frameworks de teste para validar o contrato.
  - Comparação de payloads com os esquemas definidos no Swagger.
- SBET (Session-Based Exploratory Testing)
  - Realização de sessões de testes exploratórios com tempo definido e foco em objetivos específicos.
  - Documentação das descobertas durante a sessão.
- State Transition (mudança de estado)
  - Testar as transições de estado dos recursos.
  - Exemplo: Um carrinho pode estar "ativo", "concluído" ou "cancelado". Testar as transições entre esses estados.
- Rollback e integridade transacional
  - Verificar se as operações transacionais (ex: concluir compra) são atômicas.
  - Em caso de falha, se todos os dados são revertidos ao estado anterior.

Tipos de Cenários de Teste

- Cenários positivos
  - Testes que validam o comportamento esperado da API sob condições normais e válidas.
  - Exemplo: Login com credenciais corretas, cadastro de produto válido.
- Cenários negativos
  - Testes que verificam o comportamento da API sob condições inválidas, inesperadas ou de erro.
  - Exemplo: Login com credenciais incorretas, tentar cadastrar produto sem estoque.
- Validação de contrato
  - Cenários focados em garantir que a estrutura das requisições e respostas esteja conforme a documentação.
  - Exemplo: Enviar um payload com tipo de dado incorreto para um campo.

Evidências e Exemplos

- Exemplos de payload
  - Requisições JSON de exemplo para cada endpoint.
  - Exemplo: POST /login com { "email": "fulano@qa.com", "password": "teste" }.
- Status Codes esperados
  - Códigos HTTP de sucesso (2xx), erros do cliente (4xx) e erros do servidor (5xx).
  - Exemplo: 200 OK para sucesso, 401 Unauthorized para login inválido.
- Mensagens de erro
  - Conteúdo das mensagens de erro retornadas pela API.
  - Exemplo: {"message": "Email e/ou senha inválidos"}.
- Estrutura JSON
  - Esquemas JSON esperados para as respostas da API.
  - Exemplo: GET /usuarios retornando um array de objetos de usuário.

Presented with xmind